

Distributed Systems – Lab 2

Exercise 1: Socket Programming with a Multi-Client Chat Server (Blocking)

Objective

The goal of this lab is to implement a simple chat server using socket programming in Java. Clients will connect via the command line, register with a unique name, and broadcast messages to all other connected clients through the server. Additionally, thread safety mechanisms will be employed to ensure unique client names. Future tasks will introduce artificial delays to demonstrate blocking communication.

1. Open the given **SocketChatServer** project by using File->Open projects from File system in Eclipse.
2. You can first run the ChatServer by right-clicking and selecting Run As -> Java application. Then run multiple clients by doing the same.
3. You can find the relevant lines of code for the following tasks using the TODO comments added.

Task 1:

1. Modify the server to introduce a delay (e.g., `Thread.sleep(5000)`) before sending messages to clients.
2. Observe and explain how blocking behavior affects communication.
3. Discuss how non-blocking techniques could improve responsiveness.

Task 2: Thread Safety Analysis

- What happens if multiple clients try to register with the same name at the same time?• Explain how thread safety is implemented to address the above issue: Discuss the use of synchronized blocks.
- How does the server handle a client disconnection gracefully?

Write the answers to the above questions in a text file.

Exercise 2: Asynchronous Remote Callbacks with gRPC Framework

1. Open the project **grpcAsyncTemp**.
2. The project has a server and a client. Open the src/main/java directory and run the TemperatureServer class and run multiple instances of the TemperatureClient class.
3. The temperature server simulates the reading of different temperature values from a sensor in a remote server periodically. If the temperature has varied drastically (more than 5 degrees of Celsius), the listening clients are notified. Remote callbacks are used for this purpose.

Task: Update the onNext callback function in TemperatureClient to print the updated temperature value received from the server, at the client console. There's a TODO comment in the code explaining this.

- Briefly explain why remote callbacks are suitable for this system compared to and

Exercise 3: Asynchronous communication with Message Brokers

1. For this exercise, you will have to download run Apache Active MQ Message broker. If your Java version is 11 or above, you may use the upto date version from here (<https://activemq.apache.org/components/classic/download/>). Otherwise, download an older version from here (<https://activemq.apache.org/components/classic/download/classic-05-15-14>) (zip file).
2. Extract the contents of the directory and run the activemq.bat (activemq.sh on Linux) in /bin/win64 subdirectory.

Note: If you get an error, try disabling the firewall in your computer (you may have to disable using the antivirus software). Alternatively, you can use the virtual machine.

3. If the Message broker is running successfully, you can now open the project **activem** in eclipse, and run the Receiver and Sender. Currently the communication is synchronous (blocking). That is the Receiver will wait till the sender sends a message to the queue.
4. To test this, you can run the Receiver first. It will block till there's a message on the message queue. You can open the administrator console (<http://localhost:8161/>) and open the queue (the default username and password is admin/admin). You can open the Queues tab to see the messages exchanged.
5. Follow the TODO comments in both files so that the communication is asynchronous.
6. The onMessage method on the Receiver is the callback function.

7. What can you observe when you enable asynchronous communication? What can you say about the callback function operation compared to the remote callback function in Ex2?

Submission:

Zip the updated projects as well as the answers to the questions in a text file. Name the zip file as your registration number. Upload to the courseweb link.